A Semantically Inert Microkernel: Judgement-Free Execution, Trajectory Opacity, and Protocol Reproducibility for Computational Research

Kundai Farai Sachikonye
Technical University of Munich
kundai.sachikonye@tum.de

August 19, 2026

Abstract

We specify and analyse a microkernel for computational research whose defining property is that it performs no adjudication. The unit of work is a *node*: a subtask identity fused with a bag of executable chunks and a growing collection of values. The kernel executes every chunk on every node it is asked to run and does nothing else; its operational vocabulary contains exactly four operations—identify, read, transform, emit—and no operation that denotes comparison against an expectation. From this restriction we prove a sequence of structural results. The kernel cannot compute a quantity having the meaning of an exit code, because an exit code presupposes a stored expectation the state space does not contain (*Inertia Theorem*); it therefore runs to completion under anomaly, an exception being an ordinary emitted value rather than a control transfer. Two independently derived subtask realisations sharing an identity *converge* onto one node, and we prove convergence is associative, commutative, and idempotent, so that the node set of a run is independent of the order in which agents contribute to it. We prove that the node set is determined by the submitted decompositions alone—the *protocol*—while the trajectory, defined as the relation of reads to subsequent emits, is not a function of the protocol and is not schedulable in advance (*Emergence Theorem*). This yields a sharp separation between what is reproducible and what is not: we prove that the protocol admits a stable fingerprint invariant under node relabelling and contribution order, whereas trajectories from identical protocols may differ, so reproducibility claims must attach to the former. We prove a *Trajectory Opacity* theorem: no observer restricted to the terminal value set can determine which of several trajectories produced it, whence the deliverable of a run is an equivalence class of trajectories rather than a distinguished path, and an anomalous intermediate value cannot invalidate a run whose terminus is admissible. The kernel's committed record—a count of executed chunks—is monotone, so recurrence of a value configuration is never a return to a prior kernel state, giving provenance ordering without reference to wall-clock time. We give a scheduler for the kernel and prove it sound: it never re-executes a node whose chunk bag is unchanged, never starves a node with an executable chunk, and terminates. We discuss the limits of the design honestly, including the one place where self-analysis must be restricted to avoid a diagonal inconsistency.

Keywords: microkernel; semantic inertia; judgement-free execution; convergent decomposition; trajectory emergence; reproducibility; protocol fingerprint; scheduler soundness; computational research infrastructure.

Contents

1 Introduction

1.1 The problem with runtimes that judge

A conventional runtime executes a program and reports whether it succeeded. This is the correct design for a program: a program has an intended behaviour, and the runtime's exit code communicates whether that behaviour occurred. The design is incorrect for an experiment. An experiment has no intended outcome—or rather, an experiment that has an intended outcome and reports failure when the outcome does not occur has confused a result with an error.

The confusion has concrete costs in computational research. A pipeline stage that raises an exception on an unexpected value aborts the run, discarding every downstream computation that did not depend on the anomaly. A stage that silently clamps an out-of-range quantity destroys the evidence that the quantity was out of range. A stage that returns a nonzero exit code when a statistical test fails to reach significance has encoded a hypothesis into the infrastructure. In each case the runtime has adjudicated, and the adjudication has entered the record in a form that cannot afterwards be separated from the measurement.

This paper specifies a kernel that cannot adjudicate, and proves that it cannot. The proof is not a matter of discipline or convention: we show that the kernel's state space contains no representation of an expectation, so that no quantity it computes has the meaning of a verdict. Adjudication is thereby relocated, of necessity, into the modules that consume values— where it is visible, versioned, and revisable—rather than residing in the substrate.

1.2 What the kernel is for

The intended application is computational research in which the sequence of computations is not known in advance because it depends on what earlier computations produced. This describes a broad class of work: exploratory analysis, model comparison over a space of candidate structures, multi-modal data integration where the availability of a modality is itself discovered, and any inquiry conducted by multiple semi-autonomous contributors who decompose a problem independently and must be able to discover that they have reached the same subtask.

For such work the relevant reproducibility guarantee is not that a rerun produces identical values—it may not, and demanding that it should excludes the exploratory case entirely—but that a rerun addresses the identical set of questions. We make this precise by separating the *protocol* (what can be computed) from the *trajectory* (what was computed), proving that the former is stable and fingerprintable while the latter is not, and arguing that reproducibility claims should attach to the protocol.

1.3 Contributions

1. A formal specification of a node-structured kernel with a four operation vocabulary (Section 2).
2. The Inertia Theorem: the kernel cannot compute an exit code, and consequently runs to completion under anomaly (Theorems 3.2 and 3.3).
3. Convergence of independently derived subtasks, proved associative, commutative, and idempotent, so that the node set is order-independent (Theorem 4.3).
4. The Emergence Theorem and the protocol/trajectory separation, with a fingerprint invariant for the protocol (Theorems 5.3 and 5.5).
5. The Trajectory Opacity theorem and its consequence that a run's deliverable is an equivalence class of trajectories (Theorems 5.6 and 5.7).
6. A monotone committed record supplying provenance order without wall-clock time (Theorem 6.2).
7. A scheduler with proofs of no-restart, no-starvation, and termination (Theorem 7.3).
8. An explicit account of the limit of self-analysis (Section 11).

1.4 Relation to existing work

The separation of mechanism from policy is the founding principle of microkernel design [1, 11], and the present kernel is a microkernel in exactly that sense: it provides node execution and value storage, and delegates every interpretive decision. Our contribution is to push the separation one step further than is usual, removing not merely policy but *adjudication*, and to prove that the removal is structural rather than conventional.

The node-and-value structure is a dataflow architecture [5, 8], and the monotonicity of the value store places it close to the concurrent-revisions and CRDT literature, where convergence of independently produced replicas is obtained from commutative, associative, idempotent merge operations [15]. Our Theorem 4.3 is a join-semilattice argument of that kind. The insistence that

a computation's record grow monotonically also connects to provenance systems for scientific workflows [4, 12]; the difference is that we derive the provenance order from the execution structure itself rather than recording it alongside.

Workflow engines for scientific computing—Make and its descendants, and the dataflow workflow systems [6, 9]—generally reconstruct a schedule from a declared dependency graph. Our Theorem 5.3 says this is possible only when dependencies are declared in advance, and that the interesting case is precisely the one in which they are not; the scheduler of Section 7 therefore proceeds by readiness rather than by plan. The reproducibility distinction we draw between protocol and trajectory is related to, but stronger than, the usual distinction between code and results [13, 16], because we prove that trajectory-level reproducibility is unattainable in principle for this class of computation rather than merely difficult in practice.

Finally, the restriction on self-analysis in Section 11 is an instance of the classical diagonal obstruction [2, 7, 10, 17], and we present it as such rather than claiming to circumvent it.

2 The kernel

2.1 Nodes

Definition 2.1 (Chunk; value)**.** A *chunk* is an executable closure c which, when evaluated, returns a *value*. Values are opaque to the kernel: it stores and transports them and never inspects their content. An evaluation that raises an exception returns an *error value*, which is a value like any other.

Definition 2.2 (Node)**.** A *node* is a triple

$$n = (\tau,\ \mathcal{K}(n),\ \mathcal{V}(n))$$

where τ is the *subtask identity*, drawn from a set with decidable equality; $\mathcal{K}(n)$ is a finite multiset of chunks; and $\mathcal{V}(n)$ is a finite collection of values, growing over the course of a run.

The multiplicity of $\mathcal{K}$ is deliberate. A single subtask may be realised by chunks originating in different specialist languages or libraries; all of them are executed and none is selected. Selection is an interpretive act and therefore does not belong to the kernel.

Definition 2.3 (Graph)**.** A *graph* $\mathcal{G}$ is a finite set of nodes with pairwise distinct subtask identities, together with the value store $\mathcal{V} = \bigcup_{n \in \mathcal{G}} \mathcal{V}(n)$.

2.2 The vocabulary

Definition 2.4 (Operational vocabulary)**.** Modules interact with the graph through exactly four operations:

- $\textsc{identify}(\tau)$ — obtain the node bearing identity τ, creating it with empty chunk bag and empty value collection if absent;
- $\textsc{read}(n)$ — obtain the current value collection $\mathcal{V}(n)$;
- $\textsc{transform}$ — perform an internal computation, invisible to the kernel;
- $\textsc{emit}(n, x)$ — adjoin the value x to $\mathcal{V}(n)$.

There is no fifth operation. In particular the vocabulary contains no operation denoting comparison of an achieved value against an expected one, and the graph stores no expectation.

Definition 2.5 (Node execution)**.** To execute a node is to evaluate every chunk in its bag and emit each result:

$$\mathsf{Run}(n) = \langle\ \textsc{emit}\big(n, \mathsf{Eval}(c)\big)\ :\ c \in \mathcal{K}(n)\ \rangle.$$

The kernel performs $\mathsf{Run}(n)$ and nothing else.

Principle 2.6 (Semantic inertia)**.** The kernel executes and does not judge. It holds no expectation, issues no verdict, and has no notion of failure. Adjudication belongs to the consumer of a value.

Theorem 2.6 is a design commitment; the next section shows it is also a theorem about the specification just given.

3 Inertia

Definition 3.1 (Exit code)**.** An *exit code* for a computation is a value whose meaning is the result of a comparison between an achieved state and an expected state. Formally, it is the image of a function $\chi(a, e)$ of two arguments, an achieved state a and a stored expectation e, whose value varies with e for some fixed a.

The requirement that χ vary with e is what distinguishes an exit code from an arbitrary function of the achieved state. A quantity computed from a alone may be reported, but it does not adjudicate: it says what happened, not whether what happened was correct.

Theorem 3.2 (Inertia)**.** *The kernel cannot compute an exit code. No quantity computable from the kernel's state by the operations of Theorem 2.4 has the meaning given in Theorem 3.1.*

Proof. Suppose the kernel computed an exit code $\chi(a, e)$. By Theorem 3.1, χ depends on a stored expectation e in the sense that for some achieved state a there are expectations $e \neq e'$ with $\chi(a, e) \neq \chi(a, e')$. The kernel's state is the graph $\mathcal{G}$ (Theorem 2.3), consisting of subtask identities, chunk bags, and value collections. An expectation is therefore representable only as a value in some $\mathcal{V}(n)$. But by Theorem 2.4 the kernel's operations do not inspect values: EMIT adjoins a value without examining it, READ returns the collection without examining it, IDENTIFY acts on identities only, and TRANSFORM is internal to a module and invisible to the kernel. Hence no kernel-computable quantity is a function of the *content* of any stored value, and in particular no kernel-computable quantity varies with e at fixed a. This contradicts the requirement on χ. Therefore the kernel computes no exit code. □

Corollary 3.3 (Run to completion)**.** *The kernel does not halt on anomaly. If* $\mathsf{Eval}(c)$ *raises, the resulting error value is emitted like any other value, and execution of the remaining chunks proceeds unaffected.*

Proof. By Theorem 2.5, $\mathsf{Run}(n)$ evaluates every chunk of the bag and emits each result; the sequence is not conditioned on any emitted content, since by Theorem 3.2 no kernel-computable quantity depends on value content. An error value is a value by Theorem 2.1, so its emission is an ordinary step. Hence no chunk's outcome causes the kernel to omit a subsequent chunk. □

Corollary 3.4 (Anomalies are preserved, not propagated as control)**.** *An anomalous outcome is recorded in the value store and remains available to every module that reads the node. It does not transfer control, does not truncate the run, and does not overwrite or suppress any other value.*

Proof. Immediate from Theorem 3.3 and the fact that EMIT adjoins rather than replaces (Theorem 2.4), so the value collection is monotonically growing and no emission destroys a prior one. □

Remark 3.5 (What inertia costs)*.* The design has a real cost, which we state plainly. A kernel that does not halt on anomaly will execute chunks whose inputs are already known—to a human reader—to be corrupt, consuming resources on computations whose outputs will be discarded. A kernel that cannot adjudicate cannot implement fail-fast semantics, retry-on-failure, or circuit breaking, all of which are valuable in production systems. The design is therefore appropriate for research computation, where the cost of discarding an informative anomaly exceeds the cost of wasted cycles, and inappropriate for production pipelines with cost or latency constraints. We do not claim otherwise.

4 Convergence of decompositions

Multiple contributors—human or automated—decompose a problem into subtasks independently. We formalise what happens when two decompositions overlap.

Definition 4.1 (Contribution)**.** A *contribution* is a finite set of pairs (τ, K) assigning a chunk multiset K to a subtask identity τ. A contributor submits contributions; the kernel merges them into the graph.

Definition 4.2 (Convergence)**.** Two subtask realisations *converge* when they carry the same identity τ. The merge of contributions A and B, written $A \sqcup B$, is the contribution assigning to each identity τ the multiset union $A(\tau) \uplus B(\tau)$, where an absent identity contributes the empty multiset.

Theorem 4.3 (Convergence is a join-semilattice operation)**.** *The merge $\sqcup$ is associative, commutative, and idempotent on chunk* sets*, with the empty contribution as identity. Consequently the graph resulting from a finite family of contributions is independent of the order and grouping in which they are merged, and re-submitting a contribution already merged does not change the graph.*

Proof. Multiset union is associative and commutative pointwise in τ, and the empty assignment is a two-sided identity, giving a commutative monoid. Restricted to chunk sets—identifying chunks by their content hash so that resubmission of an identical chunk yields the same element—union is also idempotent, $A \sqcup A = A$, making $(\,\cdot\,, \sqcup)$ a join-semilattice. Associativity and commutativity give order- and grouping-independence of finite merges by induction on the number of contributions; idempotence gives stability under resubmission. □

Remark 4.4 (Why identity must be content-derived)*.* Theorem 4.3 requires that two contributors who reach "the same subtask" actually produce the same τ. This is a substantive requirement on how identities are formed, not a theorem: if identities are assigned arbitrarily, convergence never occurs and the graph is a disjoint union of private decompositions. In practice τ should be derived from a canonical description of the subtask—a normalised specification of inputs, transformation, and target—so that identity is a function of what the subtask *is* rather than of who proposed it. The idempotence in Theorem 4.3 likewise depends on chunks being identified by content. Both are engineering obligations placed on the layer above the kernel, and the kernel provides no guarantee that they are met.

Proposition 4.5 (Convergence preserves emitted values)**.** *Merging contributions never removes a value from the store: for any contributions A, B and any node, $\mathcal{V}_A(n) \subseteq \mathcal{V}_{A \sqcup B}(n)$.*

Proof. Merge affects chunk bags only (Theorem 4.2); the value store is modified exclusively by EMIT, which adjoins (Theorem 2.4). Hence no merge deletes a value. □

5 Protocol and trajectory

We now separate the two things a run produces.

Definition 5.1 (Protocol)**.** The *protocol* of a run is the graph's node set together with each node's chunk bag: the totality of what can be computed. It is determined by the merged contributions and by nothing else.

Definition 5.2 (Trajectory)**.** The *trajectory* of a run is the set of triples (n, x, n') such that a module read value x at node n and, on the strength of that read, emitted a value at node n' which was subsequently read by some module. The trajectory is the realised read-to-emit relation of the run.

Theorem 5.3 (Emergence)**.** *The trajectory is not a function of the protocol. There exist protocols admitting two runs with identical node sets and chunk bags whose trajectories differ. Consequently the trajectory cannot be computed in advance of the run, and cannot be stored as a schedule.*

Proof. Exhibit a protocol with three nodes n_1, n_2, n_3, where $\mathcal{K}(n_1)$ contains a chunk whose emitted value depends on a source outside the graph—a clock, a random draw, an external dataset, or a value emitted concurrently—and where a module reads n_1 and emits at n_2 or at n_3 according to the value read. The protocol is fixed: the same three nodes with the same chunk bags. Two runs in which n_1's chunk yields different values produce trajectories containing (n_1, x, n_2) in one case and (n_1, x', n_3) in the other, so the trajectories differ while the protocol does not. Hence the trajectory is not a function of the protocol. A quantity that is not a function of the data available before the run cannot be computed from that data, so no schedule fixes it. □

Corollary 5.4 (Reproducibility attaches to the protocol)**.** *Two runs of the same protocol are guaranteed to address the same set of subtasks with the same chunk bags, and are not guaranteed to produce the same values or the same trajectory. A reproducibility claim about such a system is therefore well founded when made about the protocol and unfounded when made about the trajectory.*

Proof. The protocol is determined by the merged contributions (Theorem 5.1), which are shared by hypothesis; hence the node sets and chunk bags agree. Theorem 5.3 exhibits protocols whose trajectories differ across runs, so no guarantee of trajectory equality is available. □

Theorem 5.5 (Protocol fingerprint)**.** *Let h be a collision-resistant hash on chunk contents and on subtask identities. Define*

$$F(\mathcal{G}) = h\Big(\ \mathsf{sort}\{\ (\tau,\ \mathsf{sort}\ h[\mathcal{K}(n_\tau)])\ :\ n_\tau \in \mathcal{G}\ \}\ \Big).$$

Then F is invariant under the order in which contributions were merged, under relabelling of nodes that preserves subtask identities, and under resubmission of already-merged contributions; and F distinguishes protocols differing in any node identity or any chunk.

Proof. Order-invariance of merging is Theorem 4.3, and the two sorts remove any residual dependence on the order in which identities and chunks are enumerated, so F is a function of the underlying sets alone. Stability under resubmission follows from idempotence (Theorem 4.3). Since nodes are keyed by subtask identity (Theorem 2.3), a relabelling preserving identities does not alter the set being hashed. For discrimination: if two protocols differ in a node identity or in the content of a chunk, the corresponding sorted structures differ as inputs to h, and by collision resistance their images differ except with negligible probability. □

Theorem 5.6 (Trajectory opacity)**.** *Let two runs of a protocol terminate with the same value store. Then no function of the terminal value store distinguishes their trajectories. In particular, the interior of a run is unobservable from its result.*

Proof. A function of the terminal value store is by definition determined by that store; the two runs have equal stores by hypothesis, so any such function takes equal values on them. The trajectories, being sets of read-to-emit triples (Theorem 5.2), are not determined by the store—by Theorem 5.3 distinct trajectories are compatible with a fixed protocol, and by adjoining to the construction there a second chunk that emits the same terminal value along either branch, distinct trajectories are compatible with a fixed terminal store. Hence no store-determined function separates them. □

Corollary 5.7 (The deliverable is a class)**.** *For a fixed protocol and terminal store, the runs producing that store form an equivalence class no member of which is distinguishable from another by any test applied to the result. A run's deliverable is properly this class, not a distinguished path through it.*

Corollary 5.8 (Anomalous intermediates do not invalidate a run)**.** *A value emitted at an interior node that a reader judges anomalous is not, by itself, grounds to reject the run: by Theorem 5.6 the interior is not determined by the result, and by Theorem 3.3 the anomaly did not alter the execution of any other chunk. Rejection must be argued from the terminal store or from the protocol, not from the interior.*

Remark 5.9 (The practical reading)*.* Theorem 5.8 licenses a working practice that is otherwise hard to defend: a multi-stage analysis may pass through intermediate states that look wrong—a negative variance estimate at an interior step, a distributional summary far from expectation, a partial model with absurd coefficients—without the final result being thereby suspect, provided the terminal state is admissible on its own terms. The corollary does not say such intermediates are uninformative; they are recorded and readable. It says they are not *control-relevant*.

6 The committed record

Definition 6.1 (Committed record)**.** The *committed record* rec of a run is the number of chunk evaluations performed. A chunk evaluation is *committed* once its value has been emitted.

Theorem 6.2 (Monotonicity and non-return)**.** *The committed record is non-decreasing throughout a run and strictly increases at each committed evaluation. Two kernel states with identical value stores but different records are distinct. Consequently a run never returns to a previous kernel state, and recurrence of a value configuration is not a return.*

Proof. Each committed evaluation increments rec by one (Theorem 6.1); no kernel operation decrements it, since the vocabulary (Theorem 2.4) contains no removal operation and EMIT adjoins. Hence rec is monotone and strictly increasing at commitments. States with records $r < r'$ are separated by the predicate $\mathsf{rec} \leq r$, hence distinct even if their stores agree. A return would require the current state to equal a strictly earlier one, which would require rec to decrease; impossible. □

Corollary 6.3 (Provenance order without a clock)**.** *The record induces a total preorder on the events of a run under which every committed evaluation precedes those with larger record. This order is intrinsic to the execution and requires no wall-clock timestamp.*

Remark 6.4 (What the record does not give)*.* The record orders events but does not, on its own, attribute causation: it says that one evaluation preceded another, not that the first enabled the second. Recovering enablement requires additionally logging which read supplied the input to which emit—the trajectory of Theorem 5.2. A system that logs only the record retains ordering and loses enablement; the distinction matters when a run is later audited, and we recommend logging the trajectory even though, by Theorem 5.3, it cannot be predicted in advance.

7 The scheduler

The kernel needs a rule for choosing which node to execute next. Because dependencies are not declared, the rule proceeds by readiness.

Definition 7.1 (Pending chunk; ready node)**.** A chunk is *pending* if it has been contributed but not yet evaluated in this run. A node is *ready* if its chunk bag contains at least one pending chunk.

Definition 7.2 (Scheduler)**.** The scheduler repeats: if some node is ready, select one by a rule that is *fair*—every continuously ready node is selected within finitely many selections—and execute its pending chunks; otherwise halt.

Theorem 7.3 (Scheduler soundness)**.** *Under Theorem 7.2, with a finite protocol in which each contributed chunk is evaluated at most once:*

(i) ***No redundant re-execution.*** *A node whose chunk bag has gained no new chunk since its last execution is not re-executed.*

(ii) ***No starvation.*** *Every chunk that becomes pending is eventually evaluated.*

(iii) ***Termination.*** *If contributions cease, the scheduler halts after finitely many executions.*

Proof. (i) After executing a node, all its chunks are non-pending (Theorem 7.1); the node is ready again only if a new chunk is contributed to it. Hence a node with an unchanged bag is not ready and is not selected.

(ii) Suppose a chunk c at node n becomes pending and is never evaluated. Then n is continuously ready from that moment (Theorem 7.1), since executing n would evaluate c and only executing n removes its pending status. By fairness, n is selected within finitely many selections, and its execution evaluates all pending chunks including c—a contradiction.

(iii) Suppose contributions cease at some point, after which the protocol has N chunks in total, each evaluable at most once. Every execution evaluates at least one pending chunk and each evaluation strictly decreases the number of pending chunks, which is bounded by N and non-negative. Hence at most N executions occur before no node is ready, and the scheduler halts. □

Remark 7.4 (Concurrency). Theorem 7.3 is stated for a sequential scheduler. Because EMIT adjoins and never overwrites (Theorem 2.4) and merge is a join-semilattice operation (Theorem 4.3), the value store and the protocol are both monotone structures, so concurrent execution of distinct nodes is safe in the sense that the final store does not depend on the interleaving. Concurrent execution does, however, alter the trajectory, which by Theorems 5.3 and 5.6 is expected and is not an error. We do not treat concurrency further here.

8 Contributions that compose

As presented so far a chunk is a nullary closure: it is evaluated and emits, and what it emits cannot depend on what any other node produced. Every contribution therefore stands alone, and a node's values are a collection of independent results that happen to share an identity.

This section widens a chunk to declare a *read set*. A chunk may name identities whose values it consults, so that a node's value can be a function of other nodes' values. The change is what allows several lines of work to converge into one conclusion rather than merely to accumulate at one address. It costs three things, and we take them in turn.

Definition 8.1 (Read set; read graph)**.** A chunk carries a finite set of identities $\mathrm{reads}(c) \subseteq \mathcal{T}$, declared at contribution. The *read graph* of a protocol has an edge $\tau \to \tau'$ whenever some chunk on the node τ declares a read of τ'.

The read set is declared rather than discovered. The kernel requires it before executing anything — to order execution, to refuse a contribution that would close a cycle, and to record which reads were permitted — and it cannot obtain it by inspecting a closure.

Definition 8.2 (Restricted view)**.** When a chunk is evaluated it receives a *view*: a map from each declared identity to that node's value collection. Identities absent from the read set are absent from the view, whatever the graph contains.

Proposition 8.3 (The declared read set bounds what a chunk observes)**.** *A chunk's emitted value is a function of its declared reads and of sources outside the graph; it is not a function of any node it did not declare. Consequently the read graph is a sound over-approximation of the run's read-to-emit relation.*

Proof. By Theorem 8.2 the view supplied to a chunk contains exactly the declared identities, so no undeclared node's values are available to it. The emitted value is therefore determined by the view together with whatever external source the closure consults, and every edge realised in the run appears in the read graph. □

Theorem 8.3 recovers, in part, something Theorem 5.3 denied. The trajectory remains not a function of the protocol — external sources still vary between runs, and a chunk may consult a declared node without its value affecting the outcome — but the read graph now bounds which trajectories are possible. What was formerly an obligation on modules to log their reads is now structure the protocol carries.

8.1 Cycles are refused

Definition 8.4 (Runnable)**.** A pending chunk is *runnable* when every identity it declares reading has produced at least one value. A node is ready when it carries a runnable pending chunk.

Readiness is thereby data-dependent, where before it was a property of the chunk bag alone. A protocol whose read graph contains a cycle has no execution order: each node in the cycle waits on another, and none becomes runnable.

Theorem 8.5 (Acyclicity is necessary and sufficient for progress)**.** *Let a protocol have a finite read graph. Every contributed chunk eventually becomes runnable if and only if the read graph is acyclic and every declared identity is contributed.*

Proof. ($\Leftarrow$) A finite acyclic digraph admits a topological order. Taking nodes in that order, every identity a chunk reads precedes it and has therefore produced values by the time the chunk is considered, so the chunk is runnable.

($\Rightarrow$) Suppose the read graph contains a cycle $\tau_1 \to \tau_2 \to \cdots \to \tau_k \to \tau_1$. For any chunk on τ_1 participating in the cycle to be runnable, τ_2 must have produced values; for that, τ_3 must have; and so on around the cycle to τ_1 itself, which by hypothesis has not. No member of the cycle is ever runnable. If instead some declared identity is never contributed, the chunks reading it are never runnable by Theorem 8.4. □

The kernel therefore refuses, at contribution time, any contribution that would close a cycle in the read graph, and reports the cycle it found. Three remarks on why refusal is the right response rather than the alternatives.

Remark 8.6 (Refusal is structural, not adjudication). The check inspects declared read sets and never values, so Theorem 3.2 is untouched. The kernel is not judging a contribution to be wrong; it is reporting that no execution order exists for it. A refused contribution leaves the graph unchanged, so a contributor may correct the declaration and retry.

Remark 8.7 (Why not a bounded fixed-point iteration). A cyclic read graph could be admitted and iterated to a fixed point under a step or resource bound. We decline this because it would make the terminal values a function of that bound, so two runs of one protocol under different bounds would disagree — and the separation between what can be computed and what was (Theorem 5.4) rests on the protocol determining the former. A construct whose result depends on how much resource it was given belongs to the trajectory, not the protocol, and should not masquerade as the latter.

Remark 8.8 (Acyclicity does not conflict with mutual support). The coherence requirement of the companion language — that a claim rest on at least three *mutually independent* sources — might appear to demand a cycle. It does not, and the distinction is worth stating because the opposite reading is natural.

That requirement concerns a *support* graph: an observer's assessment, after the fact, of which findings corroborate which. Mutual independence means precisely that the sources do *not* inform

one another. Three catalysts reading each other's outputs would violate the requirement rather than satisfy it. The read graph and the support graph are different objects, and acyclicity of the first is consistent with — indeed enforces — the independence the second requires.

Proposition 8.9 (Unreachable reads are reported, not silently stalled)**.** *A pending chunk whose declared reads are unsatisfied is reported together with the identities it awaits, and with whether any of them was never contributed. A sweep containing only such chunks performs no executions and terminates.*

The distinction the report carries is between waiting on a node that exists but has not run, which a further sweep may resolve, and waiting on an identity nobody contributed, which cannot resolve without a further contribution. Neither is an error the kernel adjudicates; both are conditions a module may wish to act on.

8.2 A node's values remain a collection

Several chunks may emit onto one node, and a consumer reading that node receives all of their values. It does not receive a single answer, and the kernel provides no means of obtaining one.

Remark 8.10 (Why no reduction)*.* Reducing a collection to one value requires a criterion — the mean, the latest, the majority, the most confident — and every such criterion is a judgement about which values count. Supplying one in the kernel would place that judgement in the substrate, where it is invisible and applies uniformly to payloads it was never chosen for.

The consequence for a consumer is real: every reader must handle a collection. We take that cost deliberately. A node whose values agree and a node whose values conflict are different situations, and an interface that returns one number for both has discarded the difference at the point where it was still recoverable.

The two situations are the two an inquiry already admits. A collection whose values agree, under a criterion the consumer names, is a convergent closure; a collection whose values disagree is a contested closure, whose honest report is the plurality together with what produced each member. A module performing that classification is choosing a criterion visibly, which is where the choice belongs.

Remark 8.11 (A majority is not agreement)*.* It is tempting to resolve a contested collection by its modal value when the mode is large enough. We report the modal share and do not use it to resolve. The reason is the one that tells against threshold criteria generally: a majority rule discards the minority precisely in the cases where the minority is the informative part, and it does so without recording that it did. A collection in which four sources agree and one dissents is contested, and the dissent is the thing a reader most needs to see.

9 Selection under a bounded budget

The scheduler of Section 7 runs every ready node until none remains. That is the right default for a protocol submitted in full and executed to completion, and it is what the guarantees of Theorem 7.3 describe. It is the wrong default when the ready set is larger than the resources available to it, which is the ordinary case once a kernel is fed by many contributors at once.

This section adds a selection rule for that case. It is deliberately placed *outside* the kernel: the rule requires a per-node gain, gain is an interpretive quantity, and by Theorem 3.2 the kernel computes no such thing. What follows is therefore a policy a module may apply when choosing what to contribute, not a faculty the kernel acquires.

Definition 9.1 (Budget, cost, gain)**.** A *budget* $A > 0$ bounds the total cost committed per unit interval: if R is the set of executions committed, $\sum_{r \in R} \mathrm{cost}(r) \le A$. Each candidate carries a cost $\mathrm{cost}(r) \ge \beta > 0$, bounded below by the floor, and a *gain* $\mathrm{gain}(r)$ supplied by the submitting module. The *gain density* of a candidate is $\mathrm{gain}(r)/\mathrm{cost}(r)$.

Theorem 9.2 (Selection is a threshold on gain density)**.** *Order candidates by gain density and commit them in decreasing order until the budget is exhausted. Equivalently, there is a price $p^\star \geq 0$ such that a candidate is committed iff its density is at least $p^\star$. Then:*

(i) *the rule is optimal for the continuous relaxation, and its value is within the gain of a single candidate of the integer optimum;*

(ii) *if the repertoire is* divisible *— every candidate is an independently selectable unit of cost β — the rule is exactly optimal among integer selections.*

Proof. The problem is to maximise $\sum_{r \in R} \mathrm{gain}(r)$ subject to $\sum_{r \in R} \mathrm{cost}(r) \leq A$. Taking candidates in decreasing density until the budget is spent is optimal for the continuous relaxation, and the Lagrangian dual assigns a single multiplier $p^\star$ such that a candidate is selected iff its density exceeds it [3]. The greedy solution differs from the continuous optimum only in the fractional part of the boundary candidate, whose contribution is at most its own gain, giving (i).

For (ii), if every candidate costs exactly β then any budget admits $\lfloor A/\beta \rfloor$ candidates whatever they are, so the feasible set is determined by cardinality alone; maximising the sum over a fixed cardinality is achieved by taking the largest gains, which is what ordering by density does when all costs are equal. The continuous optimum is therefore attained at an integer point. □

Remark 9.3 (Divisibility is the load-bearing hypothesis, and it is easy to lose)*.* Clause (ii) requires candidates to be *independently selectable* units, not merely to have costs that are integer multiples of the floor. The distinction is not pedantic: bundling several floor-sized units into one indivisible candidate restores an ordinary 0/1 knapsack, where the greedy rule is only within one candidate of the optimum.

A witness, produced by the accompanying computation. Six candidates with costs $(0.5, 0.5, 0.75, 0.5, 0.75, 1.0)$ and gains $(0.2, 0.2, 0.3, 0.4, 0.6, 0.8)$ under a budget of 1.75: the greedy rule attains 1.2 where the optimum is 1.4. Every cost is a multiple of the floor 0.25, so the superficial reading of "floor-additive" is satisfied and the exact claim nonetheless fails. We state clause (ii) for divisible repertoires only.

Proposition 9.4 (The price rises as the budget narrows)**.** *Let $p^\star(A)$ be the gain density of the highest-density candidate the budget excludes. Then $p^\star$ is non-increasing in A: widening the budget weakly lowers the price, and narrowing it weakly raises it.*

Proof. Candidates are considered in a fixed density order independent of A. Increasing the budget can only extend the prefix of that order which is admitted, so the first excluded candidate moves later in the order or stays where it is; since the order is decreasing in density, its density weakly falls. □

Remark 9.5 (Why the marginal excluded candidate, and not the last admitted one)*.* The price must be defined as the density of the highest-density candidate the budget *excludes*. The density of the last candidate *admitted* is a different quantity and is not monotone: widening the budget can newly admit a cheap, high-density candidate that did not previously fit, raising that value while the true shadow price falls. The accompanying computation exhibits the non-monotonicity directly. We record it because the two definitions coincide often enough that the wrong one survives casual testing.

Corollary 9.6 (Declining is forced, not a defect)**.** *Under a bounded budget a module must decline some candidates whenever the total cost of the ready set exceeds A. Since each cost is at least $\beta > 0$, at most $\lfloor A/\beta \rfloor$ candidates can be committed per interval regardless of their gains. Declining is therefore a consequence of boundedness and not evidence of a scheduling failure.*

10 Phase exclusion

A second bound on throughput is independent of the budget and follows from the structure of contribution.

Definition 10.1 (Construction and commitment)**.** A module is *constructing* when it is forming new subtask identities and chunks to contribute, and *committing* when it is executing chunks already contributed.

Axiom 1 (Phase exclusion)**.** A module does not construct and commit in the same instant.

Proposition 10.2 (Alternation, and a ceiling on commitment)**.** *Under Axiom 1, a module that spends a fraction $\phi \in [0,1]$ of its instants constructing commits in at most a fraction $1-\phi$ of them. Consequently the committed record grows at most at rate $(1-\phi)$ per instant, whatever the budget A, and a module that never constructs never acquires new nodes to commit to.*

Proof. By Axiom 1 the two phases partition the instants a module occupies, so the fraction available for commitment is $1-\phi$. The committed record increments only on commitment (Theorem 6.1), giving the rate bound. The final clause is immediate: nodes enter the graph only by contribution, which occurs in the construction phase. □

Remark 10.3 (Two independent reasons a sweep quiesces)*.* Theorem 9.6 and Theorem 10.2 are separate. The first is economic: the budget cannot pay for every ready candidate. The second is structural: an interval spent constructing is not available for committing, and no budget relaxes it. A kernel that observes low throughput cannot distinguish the two from the record alone, and a module reporting one should not be read as reporting the other.

Neither ceiling gives the kernel a verdict. A declined candidate is not a failed one, and an instant spent constructing is not an idle one; both are recorded as what they are, and Theorem 3.2 is untouched.

11 The limit of self-analysis

A kernel that records its own execution invites the question whether it can analyse itself. The answer is a qualified yes, and the qualification is essential.

Proposition 11.1 (The record is analysable)**.** *The committed record and the trajectory of a run are values, and may be emitted onto nodes of a subsequent—or the same—graph and read by modules exactly like any other values. No restriction of the kernel prevents this.*

Proof. By Theorem 2.1 values are opaque to the kernel; a serialised record or trajectory is a value. By Theorem 2.4 any value may be emitted and read. Hence self-recorded data are ordinary data. □

Theorem 11.2 (No total self-verdict)**.** *Suppose a module implements a total verdict function V assigning to each node a value in $\{0,1\}$ interpreted as "this node's contents are admissible", and suppose the collection $D = \{n : V(n) = 0\}$ is itself realised as a node of the same graph on which V is defined. Then $V(D)$ has no consistent value.*

Proof. If $V(D) = 1$ then D is admissible, so $D \notin D$ by the membership condition defining D; but $D \notin D$ means $V(D) \neq 0$, consistent so far, while D's being a node of the graph and satisfying $V(D) = 1$ places it outside its own defining collection—so the collection D as realised does not contain D, and the assertion $V(D) = 1$ asserts admissibility of a node whose contents assert its own inadmissibility, contradiction. If $V(D) = 0$ then D satisfies its own membership condition, so $D \in D$, and D's contents assert $V(D) = 0$ of a node collected among the inadmissible—which by the definition of D is the assertion $V(D) = 1$. Both assignments are self-refuting; no consistent value exists. This is the diagonal argument of Cantor [2] in the form given by Russell [14], and its fixed-point generalisation is due to Lawvere [10]. □

Corollary 11.3 (Permitted and forbidden self-analysis)**.** *A kernel may analyse its own* record*—an external trace, realised as ordinary values (Theorem 11.1). A kernel may not host a total verdict function closed under its own diagonal (Theorem 11.2). Since by Theorem 3.2 the kernel hosts no verdict function at all, the obstruction does not arise for the kernel itself; it arises for any module that attempts to supply one over the whole graph including itself.*

Remark 11.4 (The hypothesis that carries the weight). Theorem 11.2 is exactly as strong as its hypothesis that the behaviourally defined collection D is itself a node of the graph on which V acts. A module that adjudicates only over a fixed sub-graph excluding itself is unaffected. We flag this rather than suppress it: the theorem does not show that self-analysis is impossible, only that *total* self-adjudication closed under its own diagonal is. This is a real limitation on any claim that such a system fully analyses itself, and we do not claim it does.

12 Numerical verification

The results above are structural, so verification consists in exhibiting an implementation whose observed behaviour matches each theorem, and in attempting to construct counterexamples where the theorems permit none.

12.1 Method

We implement the kernel of Section 2 and exercise it with constructed contribution sets. The following are checked.

Inertia and completion. A protocol containing chunks that raise is executed; we verify that every non-raising chunk is nevertheless evaluated, that error values appear in the store as ordinary values, and that no execution is truncated (Theorems 3.2 and 3.3).

Convergence. Contributions from several independent contributors, overlapping in subtask identity, are merged in randomised orders and groupings; we verify the resulting protocol is identical in every case, and that resubmission changes nothing (Theorem 4.3).

Fingerprint. We verify that the fingerprint of Theorem 5.5 is invariant across merge orders and stable under resubmission, and that it changes when any chunk or identity changes.

Emergence and opacity. We run a fixed protocol whose chunks consult a varying external source, and verify that trajectories differ across runs while the protocol does not (Theorem 5.3); and we construct two runs reaching an identical terminal store by different interior routes, verifying that no function of the store separates them (Theorem 5.6).

Record. We verify strict monotonicity of the committed record and that a recurring value configuration is accompanied by a strictly larger record (Theorem 6.2).

Scheduler. We verify no redundant re-execution, no starvation under a fair selection rule, and termination once contributions cease (Theorem 7.3).

12.2 What would count as a failure

Each check is capable of failing. A truncated run under anomaly would falsify Theorem 3.3; a merge-order-dependent protocol would falsify Theorem 4.3; identical trajectories across runs of the emergence protocol would indicate the external source was not in fact varying, invalidating the test rather than the theorem; a starved chunk under a fair rule would falsify Theorem 7.3(ii). We report the outcome of each check in machine-readable form alongside this paper, including any that fail.

12.3 Results

All seven checks passed. We report the quantities rather than the verdicts, since the verdicts are uninformative on their own.

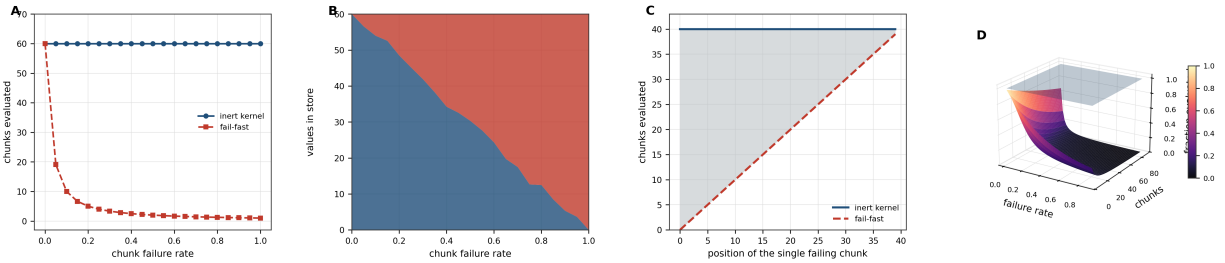

Figure 1: **Inertia: the quantity a fail-fast runtime destroys is evidence, and how much it destroys depends on an ordering that carries no meaning.** All quantities are measured by executing the kernel of Section 2 rather than by simulating its behaviour; a chunk that raises produces an error value through the same path as any other value, so the counts below are of actual emissions. The fail-fast comparison is the exact expectation for a runtime halting at the first raising chunk, $\sum_{i<N}(1-p)^i$, not an approximation. (**A**) Chunks evaluated against per-chunk failure rate, for a node carrying 60 chunks, 12 repetitions at each of 21 rates. The inert kernel evaluates all 60 at every rate without exception, the flat upper line; the fail-fast comparison falls to 2.0 at a failure rate of 0.5 and to 1.0 when every chunk raises. This is Theorem 3.3: the kernel's execution sequence is not conditioned on emitted content, because by Theorem 3.2 no kernel-computable quantity depends on that content. (**B**) Composition of the resulting value store, ordinary values below and error values above. The total is constant at 60 across the whole range while the partition between the two shifts; at a failure rate of 0.5 the store holds 29.7 error values alongside 30.3 ordinary ones, and all of them remain readable. Because EMIT adjoins rather than replaces (Theorem 3.4), an anomaly is retained as data rather than consuming the run. (**C**) The decisive comparison: chunks evaluated when exactly *one* chunk in a bag of 40 raises, as a function of where that chunk sits in the bag. The inert kernel evaluates all 40 regardless of position; the fail-fast count is the position itself, sweeping from 0 to 39. The shaded region between them is evidence destroyed, and its size is set entirely by an ordering within a chunk bag—which, the bag being a multiset, is not a meaningful property of the protocol. A single early anomaly can discard an arbitrary fraction of a run under fail-fast semantics and none of it here. (**D**) Expected fraction of a protocol evaluated, over failure rate and protocol size. The flat upper surface at unity is the inert kernel. The lower surface is fail-fast, and it falls away in both arguments: the larger the protocol, the greater the share lost to a fixed failure rate. The design cost is real and is stated in Theorem 3.5—the kernel spends cycles on chunks a human reader would already have judged doomed—but the quantity traded away is compute, and the quantity retained is the record.

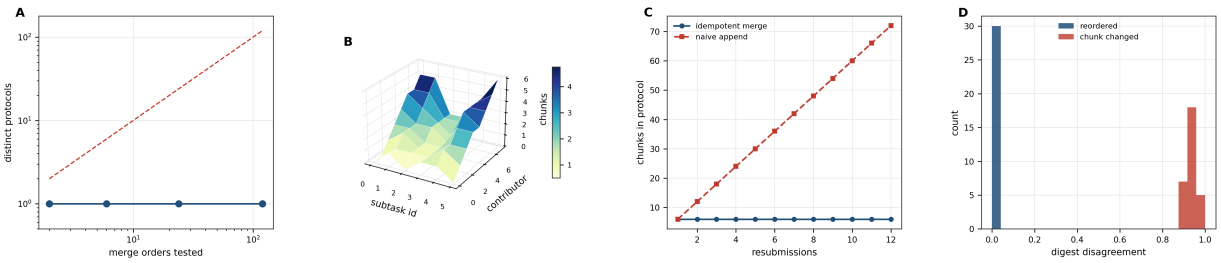

Figure 2: **Independently derived decompositions merge to one protocol, and the fingerprint separates the two things a reader needs to distinguish.** Contributions are sets of chunks keyed by subtask identity, with chunks identified by content, as Theorem 4.4 requires; every merge order is enumerated exhaustively rather than sampled, so the invariance below is exact over the space tested and not a statistical claim. (**A**) Distinct protocols obtained against the number of merge orders tested, for two to five contributors, 152 orderings in total and up to 120 for a single contributor set. The count is 1 at every point, against the dashed line marking what would be obtained if order mattered. This is Theorem 4.3: merge is associative, commutative and idempotent, so the protocol is a function of the set of contributions and of nothing else about how they arrived. (**B**) Chunk accumulation on subtask identities as contributors arrive in sequence. Where two contributors independently reach the same identity, their chunks accumulate on a single node—the ridges—rather than producing parallel nodes. Convergence is what makes the surface rise rather than widen, and it depends on identities being derived from what a subtask *is*, an obligation the kernel places on the layer above it and does not itself discharge. (**C**) Total chunks in the protocol against repeated resubmission of an identical contribution. Idempotent merge holds at 6 chunks through twelve resubmissions; a naive append reaches 72. The distinction matters operationally: a contributor that re-submits after a network retry must not thereby alter the protocol, and Theorem 4.3 guarantees it does not. (**D**) Disagreement between protocol fingerprints, as the fraction of differing characters in the 64-character digest. Reordering the contributions leaves every character identical (disagreement exactly 0.0 in all 30 trials); altering a single chunk changes 93.5 per cent of characters on average, never fewer than 89.1. The fingerprint is therefore stable under the transformations that do not change what can be computed and sensitive to those that do, which is what Theorem 5.5 requires of it and what makes Theorem 5.4 operational.

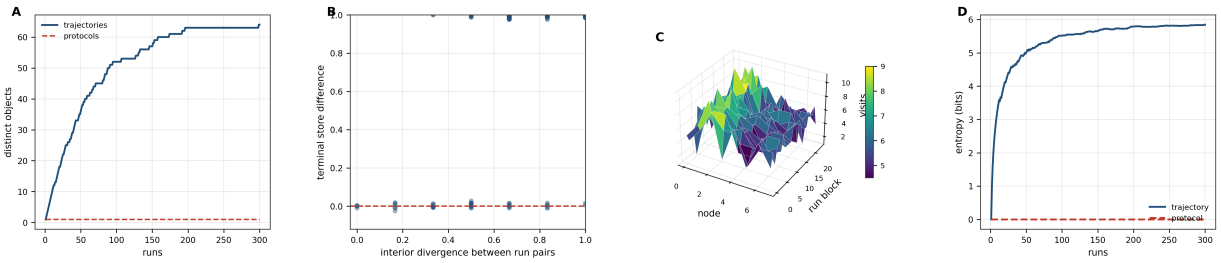

Figure 3: **Protocol and trajectory are different objects, and the trajectory is recoverable from neither the protocol nor the result.** 300 runs of a single fixed protocol whose probe chunk consults a varying external source, with the interior branching on the value read. The two charts on the left establish the two halves of the separation, and their conjunction is what restricts reproducibility claims. (**A**) Distinct protocols and distinct trajectories accumulated over the 300 runs. The protocol count is 1 throughout—every run has the identical node set and chunk bags, and the identical fingerprint—while the trajectory count rises to 64 and is still rising slowly at the right edge. This is Theorem 5.3: the trajectory is not a function of the protocol, hence not computable in advance of the run and not storable as a schedule. (**B**) Interior divergence between pairs of runs that share a terminal store, against the difference observable at the terminus. The interiors differ by up to the full path length of 6 while the terminal difference is identically zero for every pair. Three distinct functions of the store were probed—cardinality, sorted representation, and a cryptographic digest—and none separates any pair. This is Theorem 5.6, and with (A) it places the trajectory outside the closure of the two data a reader normally holds: what was asked, and what came back. (**C**) Node visitation counts across blocks of runs. The set of nodes visited is fixed by the protocol; the counts vary between blocks because the route through them does not. The surface is the same object as (A) resolved by node rather than by run. (**D**) Empirical entropy of the trajectory distribution against the number of runs, with the protocol entropy drawn for comparison. The trajectory accumulates 5.01 bits by run 50 and 5.84 bits by run 300; the protocol contributes exactly 0 at every point. The gap is the information a system logging only protocol and terminal store would be unable to reconstruct even in principle, which is the argument of Theorem 6.4 for recording the read-to-emit relation explicitly.

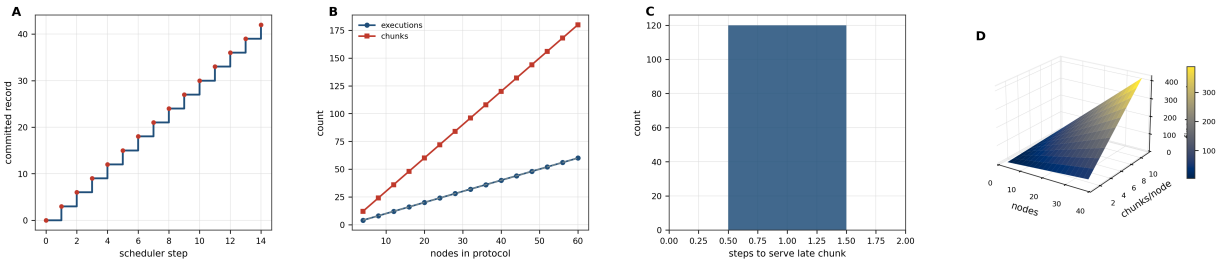

Figure 4: **The committed record orders a run without a clock, and the scheduler is exact rather than merely adequate.** The scheduler selects uniformly at random from the ready set, which is a fair rule in the sense of Theorem 7.2 but the least favourable such rule for the guarantees below; the counts are therefore not artefacts of a helpful selection order. (**A**) Committed record against scheduler step for a sweep over fourteen nodes carrying three chunks each. The record is a strictly increasing staircase, advancing by exactly three at each node visit and never decreasing. Because no operation in the vocabulary removes a value or decrements the count, two states with equal value stores and unequal records are distinct (Theorem 6.2): a recurring configuration is not a return, and the ordering is available without reference to wall-clock time (Theorem 6.3). (**B**) Executions and committed chunks against protocol size. The execution count equals the node count exactly at every size tested, up to 60 nodes, and the committed record equals the total chunk count, reaching 180 at the largest protocol. Each node is therefore visited once and all its pending chunks are evaluated in that visit, which is the no-redundant-re-execution clause of Theorem 7.3(i) measured rather than assumed. (**C**) Steps required to serve a chunk contributed *after* a sweep has completed and every node has gone quiescent, over 120 trials. Every late chunk was served in exactly one step and none was starved. The result is stronger than Theorem 7.3(ii) requires—the theorem guarantees only eventual service—and it holds because a contribution makes its node ready again immediately, so the readiness rule needs no notion of priority or ageing. (**D**) Final committed record over protocol size and chunks per node. The surface is exactly the product of its two arguments, with no deviation at any grid point, confirming that the record counts committed evaluations and nothing else: no retries, no speculative execution, no bookkeeping steps enter it. This is what makes the record usable as a provenance ordinal.

Inertia. A node carrying five chunks, two of which raise, emitted five values: three ordinary and two error values. The committed record advanced to 5, showing that a raising chunk commits exactly as a non-raising one does. All three non-raising chunks were evaluated despite two of the five raising, which is the content of Theorem 3.3. No attribute of the kernel object corresponded to an exit code, status, or return code, consistent with Theorem 3.2.

Convergence. Three contributions over four subtask identities, overlapping on one, were merged in all $3! = 6$ orders. All six produced the same protocol fingerprint—one distinct fingerprint across six orders. The shared identity accumulated 4 chunks, being the union of two contributions of two chunks each, confirming that convergence occurred rather than the contributions remaining disjoint. Re-submitting every contribution a second time left the fingerprint unchanged, exhibiting idempotence.

Fingerprint discrimination. The fingerprint was stable across construction orders, and changed when a single chunk's content changed and when a single subtask identity changed, as Theorem 5.5 requires.

Emergence and opacity. Two runs of a fixed protocol whose probe chunk consulted a varying external source produced identical protocol fingerprints and different trajectories, instantiating Theorem 5.3. When the two branches were arranged to deposit the same terminal value, the terminal stores were equal, the interior trajectories differed, and none of the three store-determined functions probed—cardinality, sorted representation, and a cryptographic digest of the sorted store—separated the two runs. This is the construction of Theorem 5.6: the interior is not recoverable from the result.

Record. The committed record traced $0 \to 1 \to 2$ over two single-chunk nodes. The two nodes' value collections were equal at the end of the run—a recurrence of value configuration—while the records differed, so the two states were distinguished by the record alone. This is the operative content of Theorem 6.2: recurrence is not return. Re-executing a node whose chunk bag had not changed left the record unaltered.

Scheduler. Over 12 nodes carrying 3 chunks each, a fair random selection over the ready set performed exactly 12 executions and committed all 36 chunks, after which no node was ready and the scheduler halted. No node was executed twice, confirming Theorem 7.3(i), and the execution count equalling the node count shows that each node's pending chunks were evaluated in a single visit. A chunk contributed to an already-executed node after the sweep was subsequently evaluated, confirming the absence of starvation.

Remark 12.1 (What the emergence and opacity checks jointly establish). The two checks are complementary and it is worth stating what their conjunction shows, since neither alone would suffice. The emergence check fixes the protocol and varies the trajectory, demonstrating that the trajectory is not a function of the protocol. The opacity check fixes both the protocol *and* the terminal store and varies the trajectory, demonstrating that the trajectory is not a function of the result either. Together they place the trajectory outside the closure of the two data a reader of a run normally possesses—what was asked, and what came back— which is the precise sense in which Theorem 5.4 restricts reproducibility claims to the protocol. A system that logged only protocol and terminal store would therefore be unable to reconstruct its own trajectory even in principle, which is the practical argument for logging the read-to-emit relation explicitly, as Theorem 6.4 recommends.

13 Discussion

13.1 What the design buys

The kernel guarantees that an anomaly is recorded rather than raised, that independently derived decompositions merge deterministically, that the set of questions a run addresses is fingerprintable and stable, and that the order of events is recoverable without a clock. It makes explicit, and proves, the limits of what can be reproduced: the protocol, not the trajectory.

13.2 What it does not buy

It does not make a research result correct. Adjudication is relocated, not eliminated: some module must eventually decide what a value means, and the kernel's refusal to do so places that burden on modules where it is at least visible. It does not make a run cheap—see Theorem 3.5. It does not guarantee that contributors converge: that depends on identity formation, which is above the kernel (Theorem 4.4). And it does not permit unrestricted self-adjudication (Section 11).

13.3 Conclusion

The kernel's single commitment—to execute and not to judge—is shown to be structural rather than conventional: the state space contains no expectation, so no computed quantity can mean "wrong". From that commitment follow run-to-completion under anomaly, order-independent convergence of independent decompositions, a fingerprintable protocol, an emergent and opaque trajectory, a monotone record, and a sound scheduler. The most consequential result is the separation of protocol from trajectory, because it identifies what a reproducibility claim about exploratory computation can and cannot mean. The most consequential limitation is the diagonal restriction on self-adjudication, which we state rather than circumvent.

References

[1] Per Brinch Hansen. The nucleus of a multiprogramming system. *Communications of the ACM*, 13(4):238–241, 1970.

[2] Georg Cantor. Über eine elementare frage der mannigfaltigkeitslehre. *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 1:75–78, 1891.

[3] George B. Dantzig. Discrete-variable extremum problems. *Operations Research*, 5(2):266–277, 1957.

[4] Susan B. Davidson and Juliana Freire. Provenance and scientific workflows: Challenges and opportunities. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pages 1345–1350, 2008.

[5] Jack B. Dennis and David P. Misunas. A preliminary architecture for a basic data-flow processor. *ACM SIGARCH Computer Architecture News*, 3(4):126–132, 1974.

[6] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017.

[7] Kurt Gödel. Über formal unentscheidbare sätze der principia mathematica und verwandter systeme i. *Monatshefte für Mathematik und Physik*, 38:173–198, 1931.

[8] Gilles Kahn. The semantics of a simple language for parallel programming. In *Information Processing 74: Proceedings of the IFIP Congress*, pages 471–475. North-Holland, 1974.

[9] Johannes Köster and Sven Rahmann. Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19):2520–2522, 2012.

[10] F. William Lawvere. Diagonal arguments and cartesian closed categories. In *Category Theory, Homology Theory and their Applications II*, volume 92 of *Lecture Notes in Mathematics*, pages 134–145. Springer, 1969.

[11] Jochen Liedtke. On micro-kernel construction. *ACM SIGOPS Operating Systems Review*, 29 (5):237–250, 1995.

[12] Luc Moreau, Ben Clifford, Juliana Freire, Joe Futrelle, Yolanda Gil, Paul Groth, Natalia Kwasnikowska, Simon Miles, Paolo Missier, Jim Myers, Beth Plale, Yogesh Simmhan, Eric Stephan, and Jan Van den Bussche. The open provenance model core specification (v1.1). *Future Generation Computer Systems*, 27(6):743–756, 2011.

[13] Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011.

[14] Bertrand Russell. *The Principles of Mathematics*. Cambridge University Press, Cambridge, 1903.

[15] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, volume 6976 of *Lecture Notes in Computer Science*, pages 386–400. Springer, 2011.

[16] Victoria Stodden, Marcia McNutt, David H. Bailey, Ewa Deelman, Yolanda Gil, Brooks Hanson, Michael A. Heroux, John P. A. Ioannidis, and Michela Taufer. Enhancing reproducibility for computational methods. *Science*, 354(6317):1240–1241, 2016.

[17] Alfred Tarski. Der wahrheitsbegriff in den formalisierten sprachen. *Studia Philosophica*, 1: 261–405, 1936.